

**Universidade Federal de Goiás
Instituto de Informática - Engenharia de Software
Software Concorrente e Distribuído**

Chat Online

Flavimar da Silva Almeida
Giancarlo Oliveira Silva
Luca Baccheschi Benetti
Lucas Oliveira de Souza

**Goiânia
2024**

Sumário

1. Introdução	3
2. Requisitos.....	3
2.1. Requisitos funcionais (RF)	3
2.2. Requisitos não funcionais	4
3. Relação com disciplina	5
3.1. Princípios de sistemas concorrentes.....	5
3.2. Princípios de sistemas distribuídos.....	5
4. Visões arquiteturais da aplicação:	7
4.1 Visão lógica.....	7
4.2 Visão de envio e recebimento das mensagens.....	8
4.3 Visão da implantação.....	8
4.4 Visão do desenvolvimento	9
5. Referências bibliográficas	10

1. Introdução

Uma das atividades mais recorrentes no cotidiano das pessoas, atualmente, é a troca de mensagens em redes sociais. Pessoas separadas por grandes distâncias geográficas, podem enviar um texto para outra, de maneira rápida e precisa. Além disso, existe também a possibilidade de troca de mensagens com várias pessoas ao mesmo tempo, abrangendo grupos familiares, de amigos ou até mesmo empresarial, tornando possível a interação de diversos indivíduos com o mesmo propósito.

Um software que tem como propósito trazer essa funcionalidade de comunicação, precisa levar em consideração que alguns pontos são cruciais para tornar a experiência do usuário agradável. É necessário que a comunicação seja eficiente, ou seja, que mensagens possam ser enviadas e entregues de forma instantânea, que seja fácil de usar, que as mensagens sejam disponibilizadas para consultas posteriores de forma ordenada, além de permitir que diversas pessoas possam interagir em conjunto. Com isso, o desenvolvimento de um software que envolva chat, deve tratar para que esses pontos da comunicação sejam atendidos.

O desenvolvimento de uma solução de comunicação direta entre duas pessoas, ou um grupo, abrange diversos conceitos da computação concorrente e distribuída. Além disso, um software desse tipo, requer que usuários sejam cadastrados, para que possam ser identificados e suas informações de cadastro, assim como o armazenamento e disponibilização de mensagens sejam realizados de forma sequencial e ordenada, conforme citado anteriormente.

Levando em consideração todos esses pontos apresentados, o presente trabalho tem como objetivo apresentar o desenvolvimento de um sistema que traz consigo um chat online, que permite aos usuários realizarem cadastro e enviar mensagens para outros usuários, assim como ingressar em um novo grupo ou grupo existente, interagindo da melhor forma possível. Para que esse software atenda aos requisitos mínimos e torne a experiência do usuário agradável, evitando problemas de concorrência, distribuição e dificuldade no compartilhamento de mensagens, o desenvolvimento aborda os diferentes conteúdos da disciplina de software concorrente e distribuído.

O Software desenvolvido encontra-se disponível no seguinte link: <http://18.231.217.212:8080/>

2. Requisitos

O software de chat online tem como principal objetivo, proporcionar a comunicação direta entre dois usuários ou a comunicação entre grupo. Para que esse propósito seja alcançado, diversos requisitos funcionais e não funcionais precisam ser abrangidos, conforme a seguir:

2.1. Requisitos funcionais (RF)

Requisitos funcionais são aqueles requisitos que se referem às funcionalidades principais do sistema, ou seja, tudo aquilo que o sistema deverá

ser capaz de fazer para atender as necessidades do usuário. Sendo assim, sistema desenvolvido tem como requisitos funcionais principais:

- **RF1** - Deve existir uma opção para que o usuário possa realizar um cadastro no sistema através de email;
- **RF2** - O usuário pode realizar o login no sistema com email e senha cadastrada;
- **RF3** - Quando logado no sistema, o usuário pode buscar um novo usuário para enviar mensagens;
- **RF4** - O usuário pode enviar mensagens para o outro usuário selecionado;
- **RF5** - O usuário pode criar um grupo para interagir;
- **RF6** - O usuário pode ingressar em um grupo já existente;
- **RF7** - Deve existir a possibilidade de conversa simultânea e coordenada dos diversos membros de um grupo;

2.2. Requisitos não funcionais

Requisitos não funcionais são aqueles que definem como o sistema executará as funções principais, garantindo as qualidades que o sistema deve ter para atender as necessidades dos usuários. Dessa forma, sistema a ser desenvolvido tem como requisitos não funcionais:

- **RNF1** - Será gerada uma imagem docker da aplicação, possibilitando o uso de containers;
- **RNF2** - O sistema será hospedado em nuvem (AWS), utilizando serviços para hospedagens de imagens docker;
- **RNF3** - O sistema utilizará filas (serviço AWS SQS) para o envio de email para cadastro de usuário;
- **RNF4** - O sistema utilizará serverless (serviço lambda) para o envio de email para cadastro de usuário;
- **RNF5** - Haverá uma aplicação web ASP.NET MVC, frontend e backend na mesma solução, com Javascript, HTML e CSS no front;
- **RNF6** - Haverá uma aplicação worker nodeJs responsável por receber o evento de cadastro e enviar o email;
- **RNF7** - Será utilizada comunicação com o frontend por meio de websockets, para que as imagens sejam refletidas em tempo real;
- **RNF8** - Será utilizado o banco de dados Sql Server, por meio do serviço RDS da AWS.

3. Relação com disciplina

O Chat Online garante a conversa em dupla ou em grupo, em tempo real. Para que isso aconteça da forma correta, existem diversos princípios referentes a computação concorrente e distribuída, que foram abordados na disciplina de Software Concorrente e Distribuído, conforme a seguir:

3.1. Princípios de sistemas concorrentes

Um sistema é dito concorrente quando múltiplas operações ou processos são executados ao mesmo tempo, onde esses diferentes processos podem acessar os mesmos recursos, trazendo o conceito de “Compartilhamento de recursos”, requerendo mecanismos que evitem conflitos. Além disso, um sistema concorrente também trata o conceito de “Comunicação entre processos”, fazendo com que processos distintos possam se comunicar, utilizando de artifícios como filas de mensagens. Em sistemas concorrentes, também deve-se atentar ao conceito de sincronização, ou seja, muitos processos podem tentar acessar o mesmo recurso ao mesmo tempo, mas é necessário coordenar esse acesso, a fim de garantir a consistência dos dados e evitar condições de corrida. Dessa forma, esses conceitos estão totalmente atrelados ao Chat Online, e para garantir que o software funcione, foi necessário utilizar algumas ferramentas e princípios que estão relacionados a Compartilhamento de recursos, Comunicação entre processos e Sincronização, conforme a seguir:

- **Uso de semáforos** - semáforos são mecanismos usados para controlar o acesso a um recurso compartilhados por múltiplas threads, evitando a condição de corrida com a limitação da quantidade de threads que podem acessar determinado recurso ao mesmo tempo. Sendo assim, no chat online, o conceito de semáforo foi aplicado para controlar o acesso ao banco de dados. Nele, os semáforos limitam a quantidade de threads que podem acessar o banco de dados para armazenar ou recuperar mensagens, garantindo que uma mensagem enviada por uma pessoa seja armazenada na sequência correta do envio, deixando a recuperação dessas mensagens na ordem.
- **Controles de threads e tasks** - controle de threads é essencial para gerenciar a execução de múltiplas operações concorrentes. Dessa forma, na aplicação, o controle de threads foi usado para lidar com conexões individuais de usuários, recebendo e enviando mensagens, garantindo estabilidade na aplicação.

3.2. Princípios de sistemas distribuídos

Um software é dito distribuído quando ele é projetado para ser operado em diferentes dispositivos que se comunicam e colaboram para realizar tarefas de propósito comum. Para que seja operado em diferentes máquinas, algumas características têm que fazer parte da aplicação, como: é necessário que exista uma coordenação dos diversos componentes, é necessário ser tolerante a

falhas, ter suporte para que múltiplas operações sejam realizadas ao mesmo tempo, além de ser necessário que garanta escalabilidade.

O chat online, por ser um ambiente de comunicação entre diversas pessoas, é um exemplo de software distribuído, uma vez que cada pessoa que está realizando uma comunicação, possui seu próprio dispositivo, ou seja, vários aparelhos tentando se comunicar ao mesmo tempo de locais diferentes. Para que isso fosse possível, o chat online precisou ser desenvolvido atendendo os princípios de sistemas distribuídos, conforme a seguir:

- **Utilização de containers (Docker)** - containers são unidades que incluem tudo o que uma aplicação precisa para ser executada, abrangendo código, bibliotecas, dependências e bases de dados. No desenvolvimento do chat online, foi utilizado container, gerando uma imagem docker da aplicação, o que possibilita escalabilidade, disponibilidade e tolerância à falha, melhorando o desempenho da aplicação.
- **Uso de websockets** - Websockets permitem comunicação bidirecional e em tempo real entre cliente e servidor, facilitando a gestão de múltiplas conexões simultâneas. No software desenvolvido, websockets auxiliaram no envio e recebimento de mensagens e disponibilização dessas mensagens para os usuários, garantindo uma conexão aberta e contínua, permitindo a troca de mensagens instantânea com latência mínima.
- **Aplicação distribuída** - refere-se a uma aplicação que pode ser executada em diferentes máquinas, comunicando entre si para realizar uma tarefa. Na aplicação desenvolvida, existem diversos usuários, cada um com o seu dispositivo, em diferentes lugares, se comunicando ao mesmo tempo, tudo isso só é possível porque é uma aplicação distribuída.
- **Uso de conceitos serverless** - o serverless permite que um determinado código seja executado em resposta a eventos e ampliando de forma automática, eliminando a necessidade de gerenciar servidores explicitamente. No chat online, foi utilizado o AWS Lambda (serverless) para o envio de emails para cadastro de usuários na aplicação. Com isso, dependendo do número de requisições para criação de usuário o AWS Lambda escala de forma automática à medida que vão surgindo novas requisições. Sendo assim, o uso do conceito de serverless na aplicação, está diretamente relacionado a escalabilidade e um bom desempenho.
- **API RestFUL** - o conceito em si não está ligado apenas a computação distribuída, mas no que se refere ao tema, o chat online trouxe consigo um backend seguindo os padrões de uma API RestFul, o que garante a ele, atributos que são necessários na computação distribuída, como: escalabilidade, tolerância a falhas, desacoplamento e interoperabilidade.
- **Uso de filas** - o uso dessa técnica, serve para gerenciar tarefas assíncronas, permitindo que múltiplos processos sejam executados. No chat online, foi utilizado serviço de fila da AWS SQS para envio de email para cadastro do usuário.
- **Aplicação baseada em eventos** - uma aplicação baseada em eventos é aquela onde existe reação a mudanças (eventos) que ocorre dentro de uma aplicação. Basicamente, ações acontecem dentro dos sistemas que notificam métodos ouvintes que executam determinadas funções quando são acionados. O chat online acaba sendo uma aplicação que faz uso de

eventos, pois eles são usados para uma mensagem ser enviada e consequentemente os ouvintes serem notificados e então armazenar a mensagem em um banco de dados, transmiti-la para outros usuários conectados e assim atualizar a interface do usuário.

4. Visões arquiteturais da aplicação:

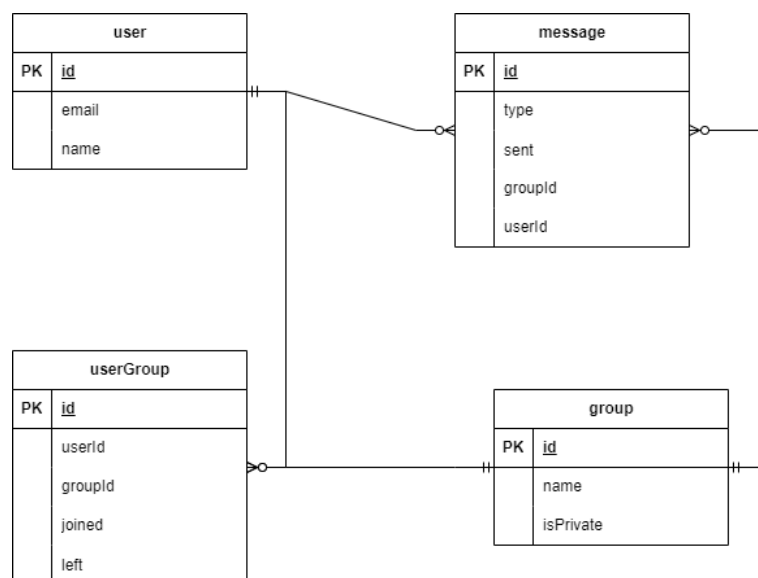
Para desenvolver a aplicação, foi levando em consideração diversas formas de representar a arquitetura, juntamente com seus componentes e comunicação, assim como seus comportamentos, e para representar isso, foi dividida em quatro visões diferentes, conforme a seguir:

4.1 Visão lógica

Na visão lógica está representado o design dos dados modelados a partir das regras de negócio e requisitos funcionais definidos anteriormente. Para essa visão, foram utilizadas as seguintes entidades:

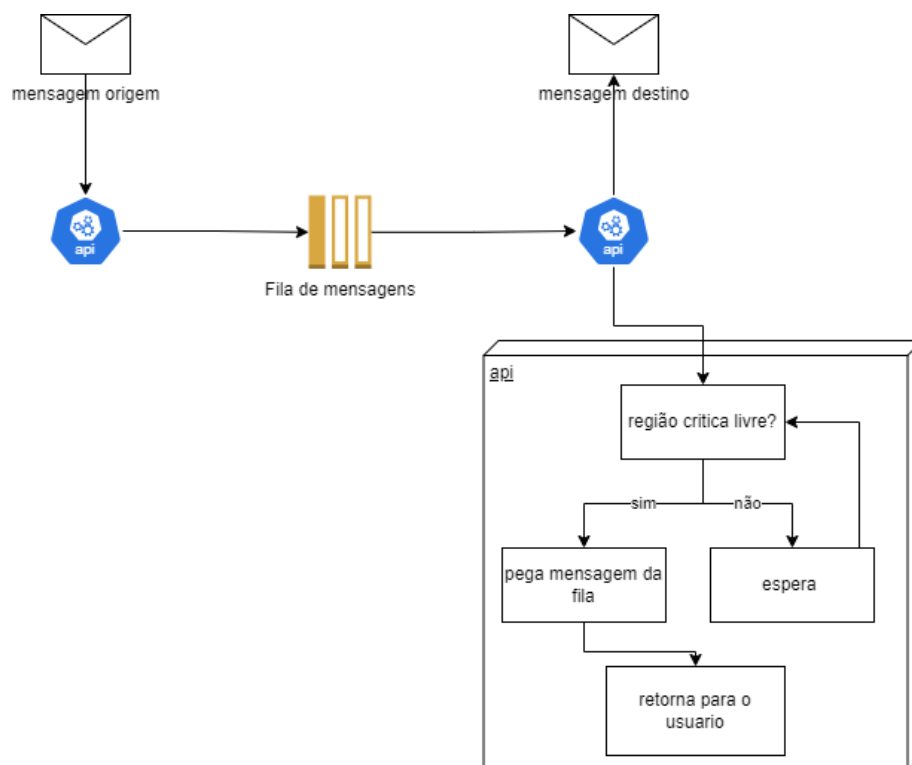
- **User:** entidade com os dados dos usuários;
- **UserGroup:** entidade de relacionamento entre as entidades User e Group;
- **Group:** entidade de agrupamento dos usuários e mensagens compartilhadas em um mesmo nicho;
- **Message:** entidade relacionado com um User em um mesmo nicho agrupada pela entidade Group;

A seguir o modelo entidade relacionamento do chat online:



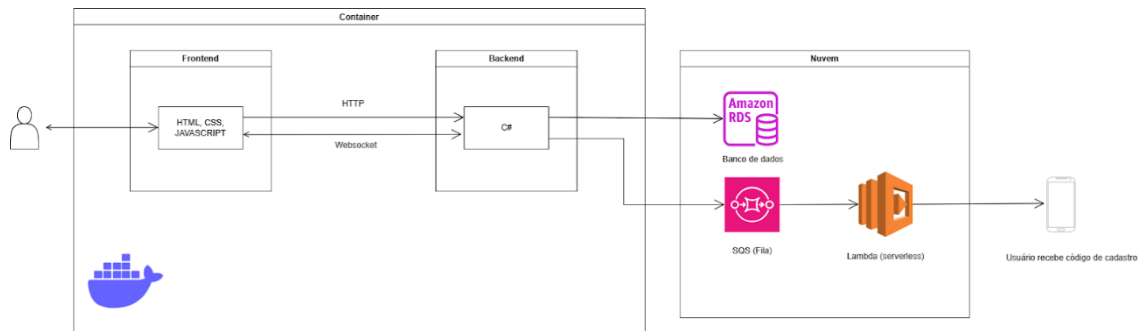
4.2 Visão de envio e recebimento das mensagens

Para enviar a mensagem, existe um fluxo por trás, que aborda a fila de mensagens e verificação de região crítica, aplicando os conceitos de computação concorrente. Antes de ser enviada, a mensagem passa pela api para gravação no banco e então é feito um enfileiramento para que sejam gravadas de forma sequencial, limitada pela região crítica. Quando são gravadas, elas são retornadas para o usuário destino e assim visualizadas. Abaixo está o fluxo que a mensagem passará para chegar no usuário destino:



4.3 Visão da implantação

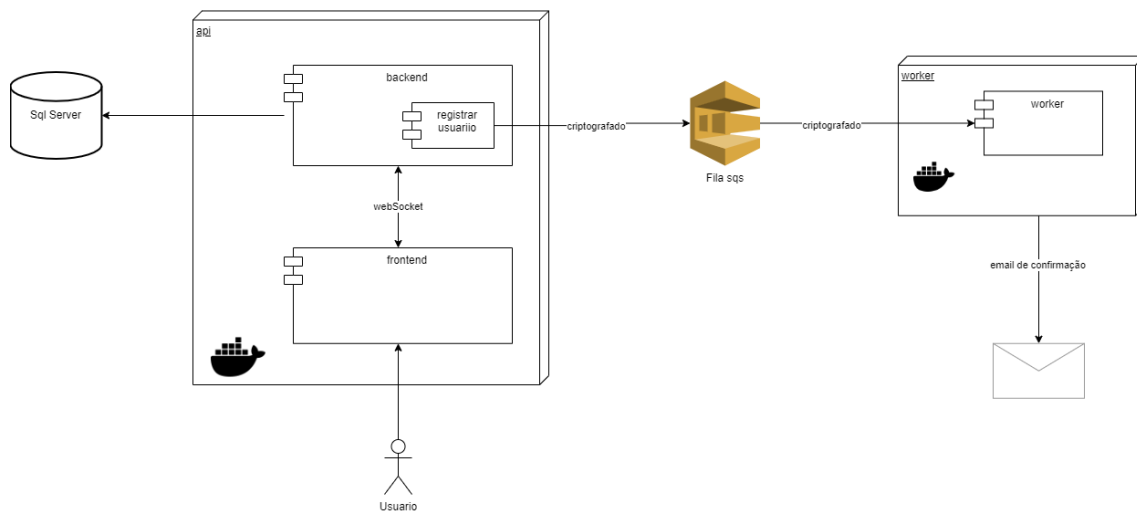
A visão de implantação (ou visão de distribuição) em uma arquitetura de software descreve como os componentes lógicos do sistema são mapeados para a infraestrutura física onde o sistema será executado. A seguir, está a modelagem da arquitetura do chat online, que mostra como acontece a comunicação entre backend e frontend, rodando em um container (docker), assim como a comunicação com a nuvem para armazenar as mensagens e para envio de email de cadastro para o usuário, utiliza SQS para enfileiramento e Lambda como serverless:



4.4 Visão do desenvolvimento

Por fim, a visão de desenvolvimento traz os principais módulos que compõem o sistema, bem como suas dependências e tecnologias. Eles são representados da seguinte forma:

- **Backend:** Aplicação em C# para gestão de usuário e gerenciamento de envio e recebimento de mensagens;
- **Banco de dados:** Banco sql server para armazenamento de dados dos usuários e mensagens;
- **Frontend:** Camada de visualização em html e css para interação dos usuários com o sistema;
- **Worker:** Aplicação em nodejs responsável pelo recebimento de novos cadastros de usuários e envio de email;
- **Fila sqs:** Serviço da aws para armazenamento de dados em fila;



5. Referências bibliográficas

Concurrency in Operating System. GeeksforGeeks, 2024. Disponível em: <https://www.geeksforgeeks.org/concurrency-in-operating-system/>. Acesso em: 19 jul. 2024.

TELES, Guilherme. **Amazon SQS: Serviço de Fila Simples da Amazon.** Medium, 2020. Disponível em: <https://medium.com/@telesguilherme/amazon-sqs-servi%C3%A7o-de-fila-simples-da-amazon-deec50071fb8>. Acesso em: 20 jul. 2024.

Criando uma AWS Lambda com Serverless Framework. Dev.to, 2020. Disponível em: <https://dev.to/feministech/criando-uma-aws-lambda-com-serverless-framework-54ph>. Acesso em: 20 jul. 2024.

AMAZON WEB SERVICES. O que é computação distribuída?. AWS, 2024. Disponível em: <https://aws.amazon.com/pt/what-is/distributed-computing/#:~:text=Computa%C3%A7%C3%A3o%20distribu%C3%ADa%20corresponde%20ao%20m%C3%A9todo,para%20lidar%20com%20desafios%20complexos>. Acesso em: 21 jul. 2024.

Arquitetura baseada em eventos. Programaria, 2020. Disponível em: <https://www.programaria.org/arquitetura-baseada-eventos/>. Acesso em: 26 jul. 2024.